



SQL ASSIGNMENT REPORT

How I Operated On Assignment 1 With SQL Commands what results looked like.

STUDENT ID	USER NAME	COURSE NAME
[29061]	[MUTANGANA Joseph]	[PL\SQL]

SECTION A: Filtering, Sorting, and Expressions

A1 Students with GPA of 3.5 or higher

QUERY

```
select first_name, last_name, email  
from students where gpa >=3.5 order by gpa desc;
```

TEST RESULTS

I executed the command, 25 rows in students table were returned and the result come order by gpa in descending order.

A2 Courses have 4 credits and belong to Computer Science department

QUERY

```
select course_name, course_code, instructor  
from courses where credits = 4 and department = 'Computer Science';
```

TEST RESULTS

The query returned 5 rows from courses table with specified column names showing courses with credits of 4



A3 Case logic to classify gpa in categories such as honors, good, and probation

QUERY

```
select first_name, last_name,  
case  
  when gpa >= 3.7 then 'Honors'  
  when gpa >= 3.0 and gpa < 3.7 then 'Good'  
  else 'Probation'  
end as Category  
from students;
```

TEST RESULTS

The query returned 50 rows from students table, the columns returned were their first name, last name, and new column called category that shows if students if in honors, good or probation

A4 Getting students whose email is domain of email.com

QUERY

```
select first_name, last_name, enrollment_date  
from students  
where email like '%@email.com' and enrollment_date >= date'2024-01-01'  
order by enrollment_date;
```

TEST RESULTS

From students table 25 rows were returned , the command filtered the email regardless what can be first letters in their email, but ends with email.com. that shows that all students have emails with email.com domain



SECTION B:

B1 Showing the department name and average salary of it's employees rounded to 2 decimal places, then sort them by average salary in descending order

QUERY

```
select d.department_name, round(avg(e.salary), 2)
as Average_salary
from departments d
join employees e on e.department = d.department_name
group by d.department_name ORDER by avg(e.salary) desc;
```

TEST RESULTS

4 rows returned

where Department of Mathematics were having high average of 82500 and lastly the Business department were last having 57444.44 as average.

B2 Showing course name and number students enrolled in course and show course with no enrollment then ordered in descending order

QUERY

```
select c.course_name , count(e.student_id) as Total_students
from courses c
left join enrollments e on c.course_id = e.course_id
group by c.course_name
order by count(e.student_id) desc;
```

TEST RESULTS

15 rows courses were returned with the number of students enrolled in it. The left join used in sql commands

ensures that the data from courses like column of courses_name show up in results even if no matching student_id in enrollments table.



B3 Sql command to show only Courses have more than students enrolled in them.

QUERY

```
select c.course_name, count(e.student_id) as More_than_five
from courses c
join enrollments e on e.course_id = c.course_id
group by c.course_name
having count(e.student_id)>5;
```

TEST RESULTS

The query return 5 rows courses from two combined tables, courses and enrollments, this query what it did, was to group courses that have their ids in enrollment table, then having comes again after each group found then calculate they counts of students who enrolled in them, and if that count is more than 5 show me those courses with their counts.

B4 Query to select students fullname and get the average of their score from enrollment table rounded to 1 decimal, but for students who have at least score in any enrolled course not all course being null valued score

QUERY

```
select s.first_name||' '||s.last_name as Full_Name,round(avg(e.score),1)
from enrollments e
join students s on e.student_id = s.student_id
group by e.student_id, s.first_name, s.last_name
having count(e.score)>0;
```

TEST RESULTS

The query displayed 50 rows of students with their fullname and average score across their enrolled courses rounded to 1 decimal after point.



B5 The query to show highest gpa, lowest, and average gpa across all students

QUERY

```
select max(gpa) as Highest_gpa, min(gpa) as Lowest_gpa, avg(gpa) as Average_gpa from students;
```

TEST RESULTS

The output came out as 3 columns, one called HIGHEST_GPA with a value of 3.9, next column showed up as LOWEST_GPA, with value of 2.7, and last column as AVERAGE_GPA, with value of 3.38. The functions make the query easier to perform and easy to understand that results in correct output and better for data analysis decisions.

SECTION C: JOINS

C1 A query to display all enrollments students and marks they got in courses they enrolled

QUERY

```
select s.first_name||' '||s.last_name as Full_Name, c.course_name, e.score  
from enrollments e  
inner join students s on e.student_id = s.student_id  
inner join courses c on e.course_id = c.course_id;
```

TEST RESULTS

The output displayed as 75 rows of students with their full name in one column, and each course they enrolled in and the score they obtained in certain course.

C2 A query to display the courses with the budget of their departments

QUERY

```
select c.course_name, d.budget  
from courses c  
left join departments d on d.department_name = c.department;
```

TEST RESULTS

The output came out as 30 rows of courses and their departments budgets



C3 a query to show employees with their manager

QUERY

```
select e.first_name as Employee_name, nvl(mngr.first_name, 'No manager') as  
manager_name from  
employees e  
left join employees mngr on e.manager_id = mngr.employee_id;
```

TEST RESULTS

80 rows were returned from self join on employees that displayed employees and their manager

C4 A query to display all students with no enrollment at all

QUERY

```
select s.first_name||' '||s.last_name as Full_Name  
from students s  
left join enrollments e on s.student_id = e.student_id  
where e.student_id is null;
```

TEST RESULTS

No rows returned means all students were enrolled

C5 A query to display departments with number employess in it.

QUERY

```
select d.department_name, mngr.first_name||' '||mngr.last_name as Manager_name,  
count(e.employee_id) as Total_employees  
from departments d  
join employees mngr on d.manager_id = mngr.employee_id  
left join employees e on d.department_name = e.department_name  
group by d.department_name, mngr.first_name, mngr.last_name;
```

TEST RESULTS

The 5 rows were returned, computer science has 60 number of employees, while Mathematics has 16 employees



SECTION D: SUBQUERIES

D1 A query to show students enrolled in PL/SQL Programming course

QUERY

```
SELECT first_name || ' ' || last_name AS Full_name
FROM students
WHERE student_id IN (
  SELECT student_id
  FROM enrollments
  WHERE course_id = (
    SELECT course_id
    FROM courses
    WHERE course_code = 'CS202'
  )
);
```

TEST RESULTS

The query executed and 10 rows were returned showing full name of the students in course of pl/sql programming, by using it's id in enrollment table.

D2 A query to display the courses whose average score is above average score of an entire enrollments

QUERY

```
select c.course_id, c.course_name
from courses c
where (select avg(e.score)
from enrollments e where e.course_id = c.course_id)
>
(select avg(e.score) from enrollments e);
```

TEST RESULTS

The returned 7 rows of courses with their course_id and course_name



D3 A query to show employees whose salary is equals to the maximum salary in their own department

QUERY

```
select e.first_name, e.last_name
from employees e
where e.salary =( select max(e1.salary) from employees e1
where e.department = e1.department);
```

TEST RESULTS

12 rows were returned with first name and last name of those employees

D4 Select course where it doesn't have any enrollment

QUERY

```
select c.course_name
from courses c
where course_id not in
(select e.course_id from enrollments e);
```

TEST RESULTS

No row returned as output of the query

SECTION E: DATA DEFINITION & MANIPULATION

E1 adding advisor_id in students table

QUERY

```
alter table students add advisor_id number references employees(employee_id);
COMMIT;
```



E2 A query increases salary by 5% to employees whose department is Engineering

QUERY

```
update employees  
set salary = salary + (salary*0.05)  
where department = 'Engineering';  
COMMIT;
```

TEST RESULTS

12 rows were affected by the changes
for example an employee called Julia
was having 48000 as her salary and after she is having 50400 as new salary.

E3 A query to delete enrollments whose status are Active, grade C, and score below 73.

A query first to check data to be affected

QUERY

```
select * from enrollments  
where status = 'Active' and score < 73 and grade = 'C';
```

RESULTS

The query returned 5 rows

A query to delete

QUERY

```
delete from enrollments  
where status = 'Active' and score < 73 and grade = 'C';
```

TEST RESULTS

The query replied that 5 rows affected and when I checked again with query above to check if they still exist, the data was not there



E4 A query to create a view called honor_roll with full name, gpa and enrollment date of the students with gpa above or equal 3.7

QUERY

```
create VIEW honor_roll(Fullname, gpa, enrollment_date) AS
select first_name||' '||last_name, GPA, enrollment_date
from students
where gpa >=3.7
order by gpa desc;
commit;

--A query to select students from a view
select * from honor_roll;
```

TEST RESULTS

A view was created successful then i select data from the view, 17 rows were returned with students ordered by gpa descending order

SECTION F: RANK()

A query to rank students based on their gpa who enrolled in same date cohort

QUERY

```
select first_name||' '||last_name as student_name, gpa,
enrollment_date, rank() over(PARTITION by enrollment_date order by gpa desc) as rank
from students;
```

TEST RESULTS

The query returned students data with new column called rank.
the way data were organize, same date was same group, and each group it has rank from 1 to last student in that same cohort ordered by gpa descending.